Neural Networks

DS6410 Machine Learning II

Lecture Roadmap

Four major topics in sequence:

1. **Architecture** — What is a neural network and how is it structured?

2. **Training** — How do we learn the weights? (Objective function, back-propagation)

3. **Regularization** — How do we prevent overfitting?

4. **Extensions** — Multiple outputs, multitask learning, and when to use neural networks

---

Presenter note: Each section builds on the previous one. By the end, students should be able to describe, train, and regularize a feedforward neural network.

Motivation: Why Neural Networks?

Key insight: Unlike classical algorithms, deep learning models continue to improve as data grows — older algorithms plateau.

- Classical ML (SVM, logistic regression): performance saturates with more data
- Neural networks: performance keeps rising with scale
- This scaling property, combined with GPU computing, has made deep learning the dominant paradigm in computer vision, speech, and NLP

Motivation: When Should You Use Neural Networks?

Neural networks are the state of the art in three major domains:

- **Computer Vision:** Image classification, object detection, segmentation
- **Speech Recognition:** Transcribing audio to text, voice assistants
- **Natural Language Processing (NLP):** Translation, sentiment analysis, large language models

When NOT to use them:

- When you have very little data (classical ML might perform better)
- When strict interpretability is required (e.g., some medical or financial regulations)
- When computational resources are highly constrained

The Building Block: Linear Prediction Functions

Before deep learning, we relied on linear models applied to hand-engineered features.

$$f(x) = w^T \phi(x) + b$$

- $\phi(x)$: Hand-crafted feature extraction (e.g., SIFT, HOG for images)
- w: Learned weights
- b: Bias term

The Limitation: Linear models cannot capture complex, non-linear relationships (like XOR) without extremely complex, manually designed features.

Instead of mapping inputs directly to outputs, we introduce an intermediate **hidden layer**.

- **Hidden nodes:** $h_1, h_2, \ldots, h_m$
- Each hidden node is a linear combination of inputs, passed through a **nonlinear activation function** σ:

$$h_j = \sigma\left(\sum_{i=1}^{d} w_{i,j}^{(1)} x_i + b_j^{(1)}\right)$$

- The final output is a linear combination of the hidden nodes:

$$f(x) = \sum_{j=1}^{m} w_j^{(2)} h_j + b^{(2)}$$

Activation Functions – Logistic Sigmoid

The logistic sigmoid is a classic activation function that squashes outputs to $(0, 1)$.

$$g(a) = \frac{1}{1 + \exp(-a)}$$

- **Pros:** Smooth, differentiable, historically popular (interpretable as a probability).
- **Cons:**
- **Vanishing gradients:** When a is very large or very small, the gradient $g'(a)$ approaches 0. This stops the network from learning.
- **Not zero-centered:** Outputs are always positive, which can cause undesirable zigzagging dynamics during gradient descent.

Activation Functions – Rectified Linear Unit (ReLU)

The **Rectified Linear Unit (ReLU)** is currently the most popular activation function for deep networks. $h(a) = \max(0, a)$

Why is ReLU preferred?

- **No vanishing gradient:** For positive inputs, the gradient is always 1. This allows for much faster and more reliable training in deep networks.
- **Computational efficiency:** It only requires a simple thresholding at zero, avoiding expensive exponential operations.
- **Sparsity:** It outputs exactly zero for negative inputs, leading to sparse representations.

Hidden Nodes as Nonlinear Learned Features

Instead of manually designing features $\phi(x)$, the neural network **learns** them.

- Each hidden node h_j can be viewed as a learned feature extractor.
- The weights $w_{i,j}^{(1)}$ determine what pattern the node looks for in the input.
- The final layer is simply a linear model applied to these learned features.

Hierarchical Learning: In deep networks (multiple hidden layers), early layers learn simple features (edges, corners), while deeper layers combine them into complex concepts (shapes, objects).

Presenter note: This is the core power of deep learning — representation learning. We shift the burden from human feature engineering to algorithmic optimization.

Neural Networks Are Universal Approximators

A neural network with just **one hidden layer** can approximate any continuous function to arbitrary precision, given enough hidden nodes.

- **Universal Approximation Theorem:**
  - Proved by Cybenko (1989) for sigmoid activation functions.
  - Later extended to other non-polynomial activation functions (like ReLU).
- **Intuition:** By combining step-like functions (or piecewise linear functions), we can build "bumps" or "rectangles" that approximate any curve.
- **Caveat:** The theorem doesn't say *how* to find the weights, nor does it bound the number of nodes required (it could be exponentially large).

Presenter note: While one layer is theoretically enough, deep networks (many layers) are often much more efficient, requiring exponentially fewer nodes to represent the same functions.

Defining the Neural Network Hypothesis Space

The hypothesis space $\mathcal{H}$ of a neural network is defined by its architecture. Before training, we must choose these **hyperparameters**:

- **Number of hidden layers**
- **Number of nodes per layer**
- **Type of activation functions**
- **Network topology**

Once the architecture is fixed, training involves finding the optimal weights w and biases b within that specific space.

Neural Network Objective Function

To train the network, we define an objective function (loss function) that measures the error between predictions and true labels.

$$J(W, b) = \frac{1}{N} \sum_{i=1}^{N} L(f(x_i; W, b), y_i) + \lambda \Omega(W)$$

- L: Loss function (e.g., squared error for regression, cross-entropy for classification)
- $\Omega(W)$: Regularization term to prevent overfitting
- λ: Regularization strength

The Challenge: Unlike linear regression or SVMs, the objective function for neural networks is **highly non-convex**. There are many local minima and saddle points.

Training: Loss Functions and Optimization

1. Choosing a Loss Function

- **Regression:** Squared Error $L(y, \hat{y}) = \frac{1}{2}(y - \hat{y})^2$
- **Classification:** Cross-Entropy $L(y, \hat{y}) = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$

2. Optimization via Gradient Descent

Update weights by moving in the negative gradient direction:

> $$W \leftarrow W - \eta \nabla_W J(W)$$

- **Mini-batch SGD:** Use small batches (e.g., 32–256) for a balance of speed and stability.
- **Adaptive Methods:** Momentum, RMSprop, Adam — adapt per-parameter learning rates to speed convergence.

Learning with Back-Propagation

To perform gradient descent, we need to compute the gradient of the loss with respect to every weight in the network: $\frac{\partial J}{\partial w_{i,j}^{(k)}}$.

Back-propagation is an efficient algorithm for computing these gradients using the **chain rule of calculus**. In particular, $\frac{\partial J}{\partial w_{i,j}^{(k)}} = \text{activation of node } i \times \delta_j$.

- **Forward Pass:** Compute the output of the network and the loss J.
- **Backward Pass:** Compute the "error signal" δ at the output layer, and propagate it backward through the hidden layers.

This avoids redundant calculations, making it computationally feasible to train networks with millions (or billions) of parameters.

Neural Networks Are Highly Expressive

Because neural networks are universal approximators and often have millions of parameters, they have an immense capacity to memorize data.

> **The Overfitting Problem:** A sufficiently large neural network can easily achieve 0% training error, even on completely random data. However, this leads to terrible performance on unseen test data.

To make neural networks useful in practice, **regularization is essential**. It constrains the network's capacity and forces it to learn generalizable patterns rather than memorizing noise.

Common Regularization Techniques we will cover:

- Weight Decay (L2 Regularization)
- Early Stopping
- Max-Norm Constraints
- Dropout

Regularization 1: Weight Decay (Tikhonov / ℓ_2)

The most common form of regularization is adding an **ℓ_2 penalty** on the network weights to the objective function.

$$J(W, b) = \frac{1}{N} \sum_{i=1}^{N} L(f(x_i; W, b), y_i) + \frac{\lambda}{2} \sum_{l} ||W^{(l)}||_F^2$$

- λ is the regularization strength hyperparameter.
- $||W^{(l)}||_F^2$ is the squared Frobenius norm of the weight matrix for layer l.

Presenter note: We typically do not regularize the bias terms b, as they do not control the complexity of the function, only its shift.

Why it works:

- It penalizes large weights, forcing the network to use all inputs a little rather than relying heavily on a few.
- This leads to smoother, less complex decision boundaries that generalize better.

Regularization 2: Early Stopping

Instead of training until the training loss is minimized, we monitor performance on a **validation set**.

- **The Process:**
  - Evaluate the model on the validation set at the end of each epoch.
  - If the validation loss stops improving (or starts increasing), stop training.
  - Return the weights from the epoch with the lowest validation loss.
- **Patience Parameter:** To avoid stopping too early due to random fluctuations, we wait for N epochs of no improvement before halting.

Why it works: It prevents the network from memorizing the training data (overfitting) by stopping before the weights become too specialized.

Regularization 3: Max-Norm Constraint

Unlike weight decay, which adds a penalty to the global objective function, the **max-norm constraint** is applied locally to each node.

- **The Constraint:** We enforce an absolute upper bound on the magnitude of the incoming weight vector for every hidden node j:

$$\|w_j\|_2 \leq c$$

- **Implementation:** After each gradient descent update, if a node's weight vector exceeds the threshold c, we project it back onto the sphere of radius c:

$$w_j \leftarrow w_j \frac{c}{\|w_j\|_2}$$

Regularization 4: Dropout

Dropout is a simple but highly effective regularization technique specifically designed for neural networks.

During Training

For each mini-batch, we randomly "drop" (set to zero) a fraction p of the nodes in a layer.

- The dropped nodes do not participate in the forward pass or back-propagation.
- The network architecture effectively changes for every single mini-batch.

During Prediction (Test Time)

We do **not** drop any nodes. We use the full network.

- Because more nodes are active than during training, the expected output magnitude would be too high.
- **Correction:** We scale the outgoing weights of the layer by $(1 - p)$ to compensate.

Dropout: Intuition Behind the Regularization Effect

Why does randomly dropping nodes improve generalization?

- **Prevents Co-adaptation:** Without dropout, nodes can develop complex co-adaptations where they rely on specific other nodes to correct their mistakes. Dropout breaks these fragile dependencies.
- **Forces Distributed Representations:** Because any node might be dropped at any time, the network cannot rely on any single feature. It is forced to distribute the representation of concepts across many nodes, making the model more robust.
- **Ensemble Effect:** Training with dropout can be viewed as training a massive ensemble of 2^N different "thinned" networks (where N is the number of nodes) that share weights. At test time, scaling the weights approximates averaging the predictions of this massive ensemble.

Presenter note: The ensemble interpretation is powerful. Ensembles almost always perform better than single models, and dropout gives us an efficient way to approximate an exponentially large ensemble.

Dropout: Does It Help? (MNIST Benchmark)

Empirical results on the MNIST handwritten digit classification task demonstrate the power of these regularization techniques.

Method	Test Error Rate
Standard Neural Network	~1.60%
Network + Dropout	~1.35%
Network + Dropout + Max-Norm	**~1.05%**

Conclusion: While dropout alone provides a significant improvement, combining it with a max-norm constraint yields the best results for standard feedforward networks on this task.

Presenter note: The max-norm constraint allows us to use a high learning rate without weights exploding, which helps the network explore the parameter space more effectively while dropout prevents overfitting.

Network Size: Bigger Is Usually Better

Question: If large networks overfit, shouldn't we just use smaller networks with fewer nodes?

> **Answer: No.** The standard practice in deep learning is to use a very large network and apply strong regularization (like Dropout and Weight Decay).

Why?

- **Optimization Landscape:** Small networks have objective functions with many bad local minima. Gradient descent often gets stuck, leading to poor training performance.
- **Easier Training:** Large, over-parameterized networks have smoother loss landscapes where most local minima are "good enough" (close to the global minimum).
- **Effective Capacity:** Regularization controls the *effective* capacity of the network, preventing overfitting while still enjoying the optimization benefits of a large architecture.

Presenter note: This is a key difference from classical ML. In deep learning, we decouple optimization (solved by making the network large) from generalization (solved by adding regularization).

Extension 1: Multiple Output Neural Networks

Unlike standard regression or binary classification that outputs a single value, neural networks can be extended to output **multiple values simultaneously**.

- **Architecture:** The final hidden layer connects to multiple output nodes; each output node has its own weights to the hidden layer.
- **Loss Function:** The total loss is typically the sum of individual losses: $J_{\text{total}} = \sum_{k=1}^{K} J_k(W, b)$.
- **Applications:** Predicting multiple continuous variables (x, y, w, h); multi-label classification (an image can be both "cat" and "outdoor").

Extension 2: Multitask Learning

Multitask Learning involves training a single neural network to perform multiple related tasks simultaneously.

- **Architecture:** The network shares the early hidden layers (the "feature extractor") across all tasks. The final layers branch out into task-specific outputs.
- **The Objective:** The loss function is a weighted sum of the individual task losses: $J_{\text{total}} = \sum_{t=1}^{T} \alpha_t J_t$
- **Why it works:**
  - **Shared Representations:** Learning to solve Task A helps the network discover features useful for Task B.
  - **Implicit Regularization:** Forcing the network to perform well on multiple tasks prevents overfitting to any single task.

Extension 3: Multiclass Classification

How do we handle classification when there are $K > 2$ mutually exclusive classes (e.g., classifying an image as a cat, dog, or bird)?

- **Architecture:** We use a multiple output neural network with exactly K output nodes.
- **Target Encoding:** The true labels are represented using **one-hot encoding**. If the true class is k, the target vector y has a 1 at index k and 0s everywhere else.
- **Output Activation (Softmax):** To interpret the network's raw outputs (logits) as probabilities, we apply the Softmax function to the final layer:

$$\hat{y}_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$$

This ensures that all outputs are between 0 and 1, and they sum to 1.

Presenter note: Softmax is a generalization of the logistic sigmoid function to multiple dimensions. It highlights the largest value while suppressing the others.

Multiclass Classification: Cross-Entropy Loss

When dealing with **mutually exclusive classes**, the network's outputs must represent a valid probability distribution over all K classes.

1. The Softmax Output Activation

Apply Softmax to the raw output logits z_k to produce probabilities $\hat{y}_k$ that are positive and sum to 1:

$$\hat{y}_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$$

2. Categorical Cross-Entropy Loss

The loss penalizes only the log-probability assigned to the **correct** class (one-hot encoding):

$$J(W, b) = -\sum_{i=1}^{N} \sum_{k=1}^{K} y_{i,k} \log(\hat{y}_{i,k})$$

- $y_{i,k} = 1$ if instance i belongs to class k, and 0 otherwise.

Extension 4: NNs as Feature Extractors

Deep neural networks learn **hierarchical representations**. Early layers learn general features (e.g., edges, textures in images), while later layers learn highly task-specific features.

> **Transfer Learning:** We can take a network trained on a massive dataset (like ImageNet) and repurpose it for a new, smaller dataset.

How it works (e.g., OverFeat approach):

- Take a pre-trained network and remove the final classification layer.
- Pass your new dataset through the network and extract the activations from the last hidden layer. These are your new **feature vectors**.
- Train a simple, linear classifier (like Logistic Regression or a linear SVM) on these extracted features.

Benefits: This approach is highly effective when you have limited data (where training a deep network from scratch would overfit) and saves massive amounts of computational resources.

Summary: Neural Networks in the ML Toolbox

1. Architecture

Universal approximators built from layers of interconnected nodes.

- Non-linear activation functions are crucial.
- Highly expressive, capable of modeling complex relationships.

2. Training

Optimized using Gradient Descent and Back-propagation.

- Loss functions tailored to the task (MSE, Cross-Entropy).
- Requires careful tuning of learning rates and initialization.

3. Regularization

Essential for preventing overfitting in over-parameterized models.

- Techniques: L2 Weight Decay, Early Stopping, Max-Norm.
- Dropout forces distributed, robust representations.

4. Extensions

Highly flexible architectures adaptable to various problem domains.

- Multiple outputs and Multitask Learning.
- Transfer Learning: using pre-trained networks as feature extractors.